université
PARIS-SACLAY

**Master SETI**
2026-2027

---

**RAPPORT DE TP**

# TP 1 : Calcul de racine carrée

---

Côme CERVANTES Joseph MOUSCADET

Nous attestons que ce travail est original,
qu'il est le fruit d'un travail commun au binôme et qu'il a été rédigé de manière autonome.

**Paris, le 07/10/2026**

# I. Architecture séquentielle comportementale

## A. Principe et graphe d'état

**Question a).** Nous cherchons à calculer la racine carrée entière d'un nombre non signé $A$ codé sur $2n$ bits, avec un résultat sur $n$ bits. La description doit être synthétisable, utiliser un seul processus et réaliser chacune des $n$ itérations en un cycle. Le résultat attendu est $Z = \lfloor \sqrt{A} \rfloor$, ce qui se vérifie par $Z^2 \leq A < (Z+1)^2$.

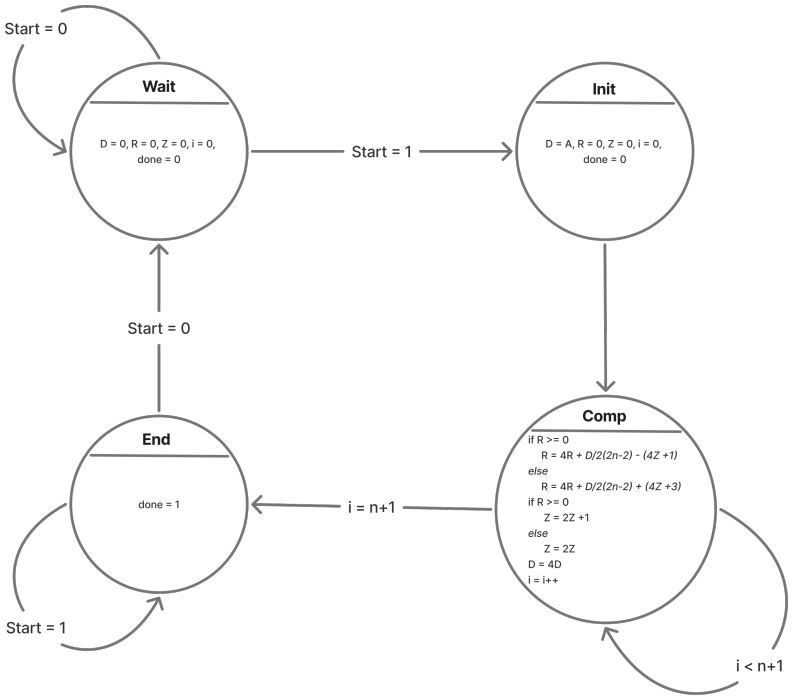



**Figure 1** – Graphe d'état utilisé pour organiser l'architecture séquentielle.

Le graphe de la figure 1 sépare l'attente, le chargement, le calcul et la fin du traitement. Il correspond aux quatre états du type énuméré utilisé dans `sqrt_seq.vhd` :

- `wait_state` : mise à zéro de $D$, $R$, $Z$, du compteur et de `done`. Le circuit attend `start = '1'` pour passer à l'initialisation.
- `init_state` : chargement de $A$ dans $D$, remise à zéro des autres données, puis passage au calcul au cycle suivant.
- `comp_state` : exécution d'une itération par front montant. L'état est conservé tant que $i < n-1$ ; la dernière itération conduit à `end_state`.
- `end_state` : `done` passe à 1. Le résultat est conservé ; le circuit reste dans cet état si `start = '1'`, sinon il retourne à l'attente.

Le reset actif à 1 impose l'état d'attente de manière asynchrone. Sur le dessin, le terme de division doit se lire $D/2^{2n-2}$ et l'incrément du compteur $i \leftarrow i+1$ ; les étiquettes $i = n+1$ et $i < n+1$ doivent être corrigées en $i = n-1$ et $i < n-1$ pour correspondre au compteur du code.

# B. Algorithme et choix des types

L'algorithme construit la racine bit par bit, en traitant les bits de $A$ par paires, depuis les poids forts. Au début d'une itération, on extrait $p = D(2n-1 \text{ downto } 2n-2)$, soit un entier entre 0 et 3. Avec les anciennes valeurs de $R$ et $Z$, on calcule :

$$R_{\text{nouveau}} = \begin{cases} 4R + p - (4Z+1), & R \geq 0, \\ 4R + p + (4Z+3), & R < 0. \end{cases}$$

Le signe du *nouveau* reste détermine ensuite le bit ajouté à la racine :

$$Z_{\text{nouveau}} = \begin{cases} 2Z+1, & R_{\text{nouveau}} \geq 0, \\ 2Z, & R_{\text{nouveau}} < 0. \end{cases} \qquad D_{\text{nouveau}} = (4D) \bmod 2^{2n}.$$

Les multiplications par 2 et par 4 sont réalisées par des décalages à gauche de un ou deux bits. L'extraction des deux bits de poids fort remplace la division entière par $2^{2n-2}$. Le décalage de $D$ élimine la paire déjà traitée et présente la suivante. Il n'est donc pas nécessaire d'utiliser un diviseur ou un multiplicateur général.

| **Élément** | **Type et taille pour $n = 32$** | **Rôle et justification** |
|---|---|---|
| `clk, rst, start, done` | `std_logic`, 1 bit | Signaux de commande et indication de fin. |
| `A` | `std_logic_vector`, 64 bits | Bus d'entrée ; converti en `unsigned` lors du chargement. |
| `Result` | `std_logic_vector`, 32 bits | Bus de sortie, relié à $Z$ par une conversion de type. |
| $D$ (signal) | `unsigned`, $2n = 64$ bits | Copie décalée de l'opérande, toujours non signée. |
| $Z$ (signal) | `unsigned`, $n = 32$ bits | Racine partielle puis résultat entier non négatif. |
| $R$ (variable) | `signed`, $n + 2 = 34$ bits | Reste partiel pouvant être négatif ; les bits supplémentaires servent au signe et aux opérations intermédiaires. |
| $i$ (signal) | `integer range 0 to n-1` | Compteur borné de 0 à 31, correspondant aux 32 itérations. |
| `curr_state` | Type énuméré à quatre valeurs | Rend explicites les phases de commande de l'automate. |

**Table 1** – Signaux et variable de l'architecture comportementale.

Le paramètre générique $n$ fixe la largeur du résultat et le nombre d'itérations ; $n = 32$ donne une entrée sur 64 bits. Le paquet `numeric_std` fournit les opérations sur `signed` et `unsigned`. Les conversions et les appels à `resize` mettent les opérandes à la largeur de $R$ avant les additions, soustractions et décalages. Contrairement à la boucle décroissante du sujet, notre compteur est croissant : l'ordre des paires traitées reste identique, car il est imposé par le décalage de $D$.

## C. Un seul processus et une itération par cycle

La commande et le calcul sont regroupés dans le processus `PCalc`, sensible à `clk` et `rst`. Hors reset, les mises à jour se font sur les fronts montants. L'affectation concurrente de `Result` à partir de $Z$ ne constitue pas un second processus de commande ; elle expose simplement le registre de résultat.

**Pourquoi une variable pour $R$ ?** Une affectation de variable avec `:=` prend effet immédiatement dans l'exécution du processus. Le second test de signe utilise ainsi le reste qui vient d'être calculé au même front d'horloge. Une affectation de signal avec `<=`, suivie du même test dans ce processus, lirait encore l'ancienne valeur. Ce choix permet de respecter l'enchaînement de l'algorithme en un seul cycle par itération.

La variable $R$ conserve néanmoins sa valeur entre deux activations du processus : elle représente ici une donnée mémorisée, et non une valeur automatiquement remise à zéro à chaque cycle. Les signaux $D$, $Z$ et $i$ sont également mémorisés, mais leurs nouvelles valeurs ne sont visibles qu'après l'exécution du processus. Les calculs du cycle utilisent donc l'ancien $Z$ et l'ancien $D$, conformément aux équations précédentes.

**Interface de commande.** L'entrée $A$ doit être stable au front exécutant `init_state`, où elle est effectivement copiée dans $D$. Elle n'est plus utilisée pendant les itérations. La sortie `Result` présente des valeurs partielles pendant le calcul ; elle doit être exploitée lorsque `done = '1'`. Maintenir `start` à 1 permet de conserver l'état de fin. Après sa remise à 0, l'automate retourne à l'attente, puis efface le résultat lors de l'exécution de cet état.

## D. Temps de calcul et organisation du testbench

Le cœur du calcul réalise exactement $n$ itérations, donc $n$ cycles. Le protocole ajoute un cycle d'initialisation et un cycle pour exécuter l'état de fin et lever `done`. À partir du front qui détecte `start` dans l'état d'attente, les délais sont donc :

$$N_{\text{Result final}} = n + 1, \qquad N_{\text{done}} = n + 2, \qquad t_{\text{done}} = (n + 2)T_{\text{clk}}.$$

Ces nombres désignent des *intervalles* d'horloge depuis le front de détection. Depuis le chargement de $A$, `done` arrive après $n + 1$ cycles. Le nombre d'itérations reste constant, quelle que soit la valeur de $A$.

Dans `testbench.vhd`, $n = 32$, le reset dure 5 ns et l'horloge a une période de 20 ns, soit 50 MHz en simulation. Le premier `start` est activé à 25 ns, détecté au front de 30 ns. Le déroulement déduit du code est :

| **Événement du premier calcul** | **Instant** |
|---|---|
| Détection de `start` | 30 ns |
| Chargement de $A = 100$ dans `init_state` | 50 ns |
| Première / dernière itération | 70 ns / 690 ns |
| Résultat final / activation de `done` | 690 ns / 710 ns |

**Table 2** – Chronologie attendue d'après le testbench et l'automate.

Le calcul itératif représente 640 ns ; le délai entre la détection de `start` et `done` vaut 680 ns. Le second opérande, 25, est présenté à 785 ns, avec un nouveau `start` à 805 ns : sa détection est attendue à 810 ns et `done` à 1490 ns. Ces durées utilisent l'horloge du testbench ; elles ne constituent pas une mesure de fréquence maximale sur FPGA.

# E. Résultats des simulations

Les captures fournies montrent quatre couples d'opérandes. Le tableau 3 compare les racines entières attendues aux résultats lisibles sur les chronogrammes. Les carrés parfaits, les petites valeurs et les valeurs non carrées permettent de vérifier plusieurs cas de fonctionnement.

| Entrée $A$ | $\lfloor\sqrt{A}\rfloor$ attendu | Résultat observé | Bilan |
|---|---|---|---|
| 0 | 0 | 0 | Conforme |
| 1 | 1 | 1 | Conforme |
| 15 | 3 | 3 | Conforme |
| 25 | 5 | 5 | Conforme |
| 100 | 10 | 10 | Conforme |
| 512 | 22 | 22 | Conforme |
| 1 194 877 489 | 34 567 | 34 567 au curseur | Conforme |
| 4 294 967 295 | 65 535 | Libellé tronqué « 6... » | À confirmer |

**Table 3** – Comparaison entre résultats attendus et lecture des captures.

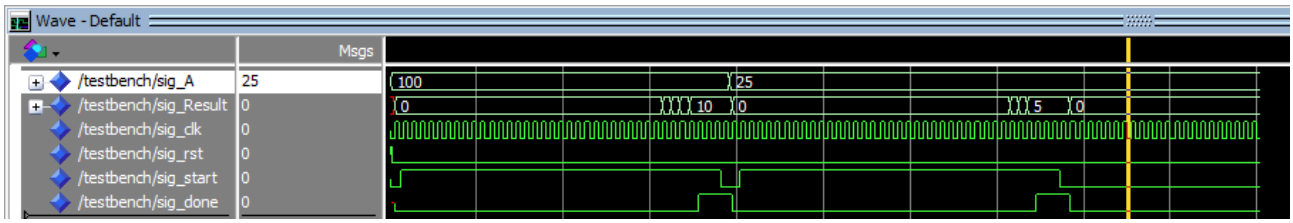



**Figure 2** – Essais $A = 100$ puis $A = 25$ : résultats 10 et 5.

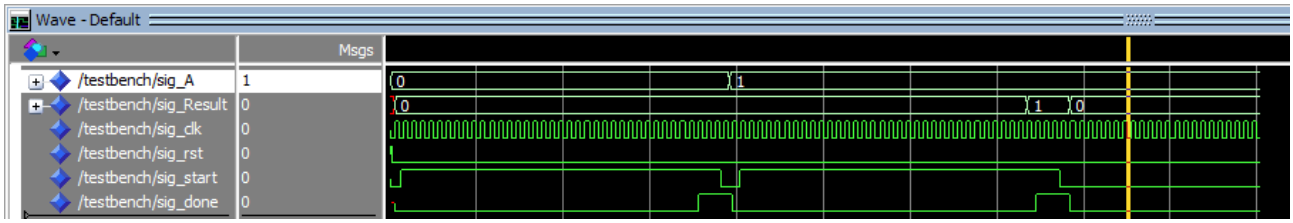



**Figure 3** – Essais $A = 0$ puis $A = 1$ : résultats 0 et 1.

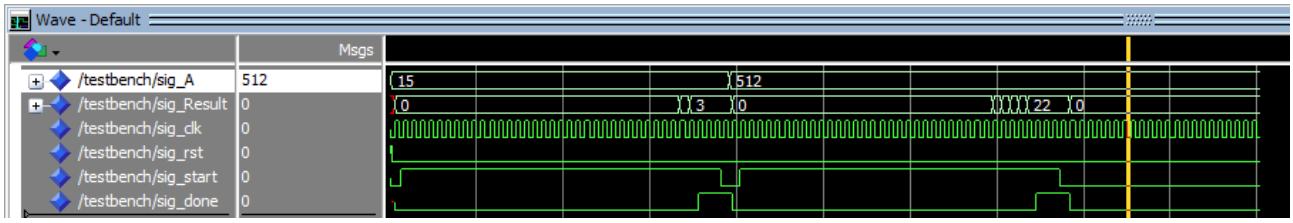



**Figure 4** – Essais $A = 15$ puis $A = 512$ : résultats entiers 3 et 22.

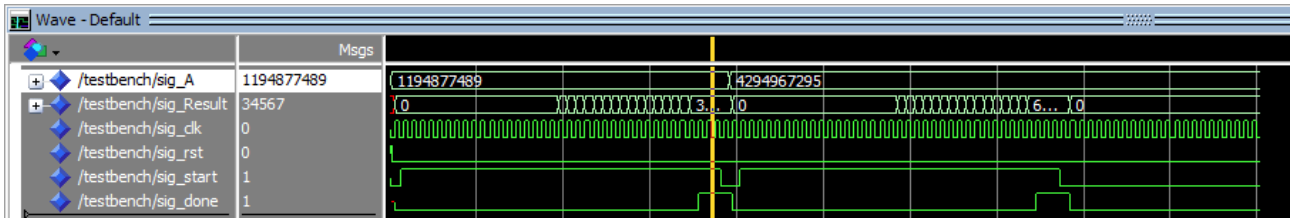



**Figure 5** – Essais sur de grandes valeurs : 34 567 est lisible au curseur pour le premier calcul ; le résultat du second est tronqué à cette échelle.

# F. Analyse des résultats et limites de validation

Les figures 2 et 3 confirment les résultats des carrés parfaits et des cas 0 et 1. Pour les valeurs non carrées de la figure 4, le circuit renvoie bien la partie entière : $3^2 = 9 \leq 15 < 16 = 4^2$ et $22^2 = 484 \leq 512 < 529 = 23^2$. Pour la première grande valeur, la figure 5 affiche 34 567 lorsque `done=1`, et $34\,567^2 = 1\,194\,877\,489$.

Les chronogrammes montrent que `Result` évolue pendant le traitement et reste stable pendant la phase de fin. Les retours à zéro entre deux calculs correspondent à l'exécution de l'état d'attente. La durée de calcul ne dépend pas de l'opérande : même les petites valeurs parcourent les 32 itérations.

**Erreur rencontrée avec un grand opérande.** La figure 6 montre un échec de compilation du testbench sur le littéral décimal `4294967295`. Le diagnostic indique que cette valeur dépasse `INTEGER'high`, soit 2 147 483 647 dans cet environnement. Le passage par `to_unsigned` ne contourne pas cette limite, car son argument doit déjà être représentable comme un entier naturel. Il s'agit donc d'un problème de construction du stimulus, et non de la largeur de l'entrée $A$, qui possède bien 64 bits.

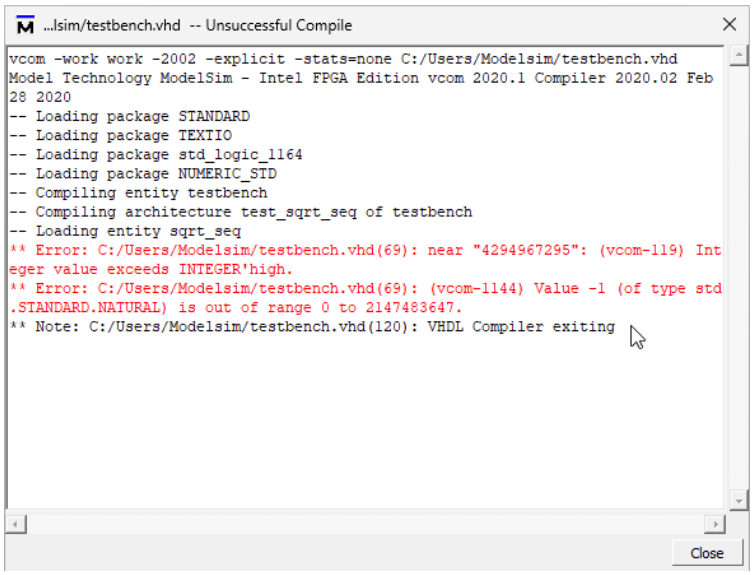



**Figure 6** – Erreur de compilation liée au dépassement du type entier dans le testbench.

Une solution consiste à affecter directement au bus de 64 bits un littéral hexadécimal sur 64 bits, `x"00000000FFFFFFFF"`, qui représente 4 294 967 295, sans conversion préalable depuis un entier décimal. La capture de la figure 5 montre bien cet opérande sur le bus d'entrée, mais ne documente pas la modification du testbench utilisée pour y parvenir. La valeur attendue est 65 535, car $65\,535^2 \leq 4\,294\,967\,295 < 65\,536^2$. Une lecture au curseur pendant `done=1`, ou une assertion, reste nécessaire pour confirmer intégralement ce dernier résultat.

**Portée de cette première validation.** Le testbench présent dans le dossier contient les essais 100 et 25 ; les autres cas sont documentés par les captures de leurs simulations. Il ne comporte pas d'assertions automatiques. Les valeurs testées restent inférieures à $2^{32}$ alors que l'entrée accepte 64 bits : elles ne couvrent donc pas toute la plage $0 \leq A \leq 2^{64} - 1$. Pour compléter la validation, il faudrait notamment tester cette borne supérieure, des valeurs autour de grands carrés parfaits et un reset pendant le calcul.

Dans le code actuel, le reset ne réinitialise directement que l'état ; $D$, $Z$, $R$, $i$ et `done` sont remis à zéro au front qui exécute ensuite l'attente. La sortie peut donc être indéfinie avant ce front au démarrage. Cette particularité doit être prise en compte par l'environnement de commande.

**Conclusion.** L'architecture réalise une itération par cycle dans un seul processus. Les sept résultats entièrement lisibles sont conformes. Le calcul nécessite $n$ cycles, et `done` est levé après $n + 2$ cycles depuis la détection de `start`. Le dernier grand résultat et la plage complète de 64 bits restent à valider. Les sources associées sont `sqrt_seq.vhd` et `testbench.vhd`.